

Отчет по мультиагентной системе

<https://github.com/paolalenti/MAS>

Быков Кирилл 11-311, Гилязов Ренат 11-311, Киселев Егор 11-311

1. ТЗ

Цель: Создание автономной системы генерации образовательного контента по запросу.

Ключевой функционал:

- Декомпозиция сложной темы на учебные модули.
- Автоматическое извлечение знаний из внешней базы (RAG).
- Формирование проверочных материалов (Quiz) в формате структурированного JSON.

Результат: Готовый к употреблению API-ответ с уроком и тестом.

2. Изоляция среды

Вариант	Принцип работы	Плюсы	Минусы
VM (Виртуальные машины)	Полная эмуляция железа (Hypervisor).	Максимальная изоляция ядра.	Огромный оверхед по RAM и CPU.
Sandbox (gVisor/Wasm)	Перехват системных вызовов или байт-код.	Безопасность выше, чем у контейнеров.	Сложность настройки GPU-пассфру.
Контейнеризация (Docker)	Изоляция через namespaces и cgroups.	Легкость, мобильность, нативный GPU.	Общее ядро с хостом.

Наш выбор: Docker + Ollama.

Почему: Для локальной разработки и MAS критична скорость обмена данными и доступ к видеокарте. Docker обеспечивает достаточный уровень изоляции для Python-зависимостей, а Ollama инкапсулирует в себе всё управление весами моделей, избавляя от конфликтов драйверов.

3. Выбор LLM

На текущий момент используется **Qwen 2.5:7b**.

Обоснование: Она показала лучший баланс между «рассудительностью» (reasoning) и способностью вызывать инструменты (tool calling) на малом количестве параметров.

Альтернативы: llama3.1:8b - ПК не вывез, недостаточно RAM; llama3.2:3b - глупая (не следует инструкциям из system prompt и не использует инструменты); qwen3.5:9b - умная, но генерирует в 5 раз медленнее.

Таким образом, **Qwen 2.5:7b** вполне нас устраивает, но если бы позволяли мощности ПК, то выбрали бы qwen3.5:9b.

4. Архитектура MAS

Мы выбрали архитектуру **Directed Acyclic Graph (DAG)** с циклами на базе **LangGraph**.

- **Зачем:** Линейные цепочки (chains) не умеют возвращаться назад. Наша архитектура позволяет агенту «сходить за знаниями», проверить их и, если результат не устраивает, повторить поиск (Reflection Loop).
- **Разделение ролей:** Это предотвращает «размытие» контекста. Архитектор разбивает тему на модули, Писатель пишет содержание для каждого модуля, Мастер тестов пишет квизы. Это чище и надежнее, чем одна LLM на все задачи.

5. Компоненты системы

System Prompts: Каждая нода имеет свою «личность» и жесткие инструкции по формату вывода.

Skills (Tools): Реализован навык `search_knowledge_base`, который связывает логику агента с физическим хранилищем.

Semantic .md: База знаний хранится в Markdown, разбивается на эмбединги, хранится в Qdrant.

Point 001e99c1-7fb0-4c8a-b3b3-70ec8f32ce7f

Payload

```
{
  "page_content": "## III. Основные Типы Данных * **Integer:** Целое _ "
  "metadata": {
    "source": "knowledge_base/sqlalchemy_guide.md"
    "title": ""
    "filename": "sqlalchemy_guide.md"
  }
}
```

Vectors:

Default vector

Length: 384

Copy

Open Graph

Find Similar

Point 01d62a9e-3590-4b9f-8afa-e66f5a48d26d

Payload

```
{
  "page_content": "** **Контейнер (Container):** Запущенный экземпляр _ "
  "metadata": {
    "source": "knowledge_base/docker_guide.md"
    "title": ""
    "filename": "docker_guide.md"
  }
}
```

Vectors:

Default vector

Length: 384

Copy

Open Graph

Find Similar

6. Система организации памяти

Мы используем гибридную схему:

- Short-term (Short-term memory):** LangGraph State. Хранит текущую историю сообщений в рамках одного прогона.
- Long-term (Persistent memory):** Векторная БД (Qdrant). Хранит статичные знания.

Вариант памяти	Недостатки
Simple Context Buffer	Забывает начало при длинных диалогах.
Summary Memory	Может потерять важные мелкие технические детали при сжатии.
Naive RAG	Часто выдает «дубликаты» (одинаковые по смыслу куски текста), что сужает кругозор агента и тратит контекстное окно впустую.

MMR-based RAG	Наш выбор. Требуется более тонкая настройка параметров (<code>fetch_k</code> , <code>lambda</code>), но обеспечивает максимальное разнообразие найденных фактов.
LLM Wiki	Сложность в реализации автоматического обновления связей между документами и риск накопления «галлюцинаций» в базе.

Обоснование выбора: Обычный «наивный» поиск (Similarity) просто находит самые похожие куски текста. Если в базе есть 10 описаний того, что такое «Docker Image», он может выдать их все сразу. Алгоритм **MMR** работает иначе: он сначала отбирает 30 подходящих кандидатов (`fetch_k`), а затем выбирает из них 10 самых разнообразных (`k`).

7. Evals

RAGAS (RAG Assessment): Оценка верности ответов (faithfulness) и релевантности найденных документов.

Faithfulness (Верность / Отсутствие галлюцинаций)

- **Что это:** Насколько ответ LLM обоснован *только* теми документами, которые выдал Qdrant.
- **Как считать вручную:** Берем ответ агента и список найденных чанков. Если в ответе есть утверждение (например, «Docker использует файловую систему AUFS»), а в найденных документах про AUFS ни слова — это минус балл.
- **Зачем это нужно:** Чтобы понять, не начинает ли модель использовать свои «общие знания» вместо нашей документации.

Answer Relevance (Релевантность ответа)

- **Что это:** Насколько ответ вообще соответствует вопросу пользователя.
- **Как считать вручную:** Бывает так, что RAG нашел правильные документы, ответ по ним верный (Faithfulness 100%), но агент ушел в дебри и не ответил на конкретный вопрос. Оценивается по шкале от 0 до 1.
- **Пример:** Пользователь спросил «Кратко про Docker hub», а модель выдала лекцию о том, что такое контейнеры и их разновидностях. Это низкая релевантность.

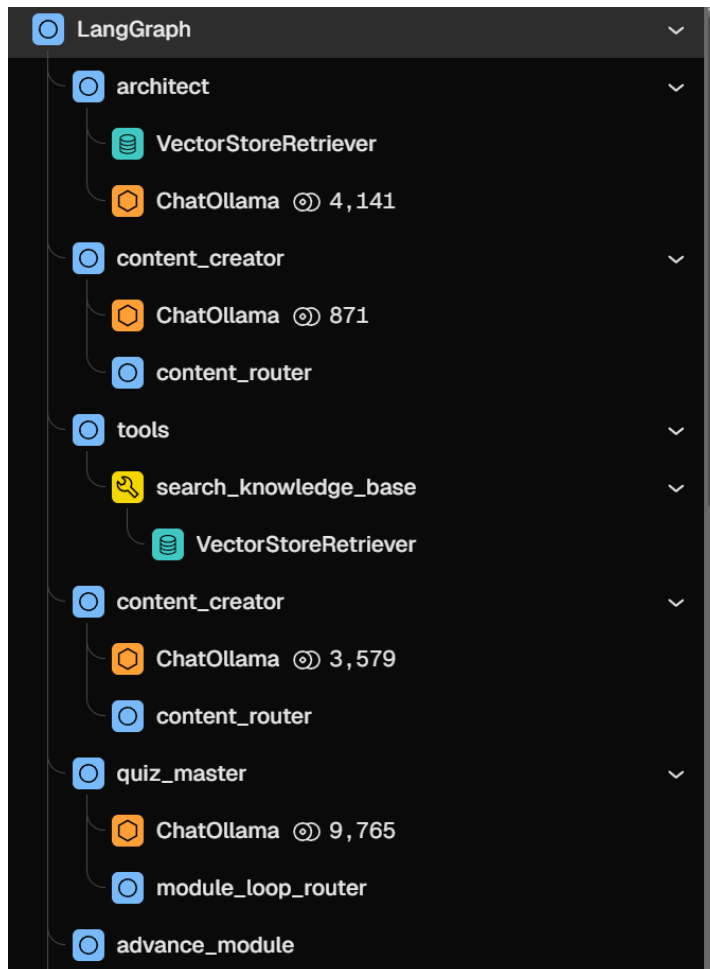
Agentic Performance:

- **Success Rate:** Процент успешных генераций JSON без ошибок парсинга.
- **Loop Count:** Сколько в среднем итераций поиска требуется агенту.

8. Observability

Для полной прозрачности системы настроен следующий стек:

- **Трейсы (Traces): Arize Phoenix.** Позволяет видеть каждый шаг графа, включая расход токенов, и Tool Call.
- **Метрики (Metrics): Prometheus + Grafana.** Собирает метрики по API (RPS, Latency).
- **Логи (Logs): Loki** собирает логи из контейнера API.



9. Немного про backend, реализованный для смысловой целостности работы MAS

Для того чтобы наше приложение имело какой-то конечный смысл и MAS не просто "генерил курсы в пустоту", мы решили добавить backend.

Мы храним данные о пользователях и сгенерированных ими курсах с тестами, пользователь имеет возможность по API взаимодействовать с ними: читать уже сгенерированное, удалять, проходить тесты. Ниже структура БД в PostgreSQL:

